

Machine Learning for Natural Language Processing

The Transformer Model in Detail



UNIVERSITY OF
GOTHENBURG

CHALMERS

Richard Johansson

`richajo@chalmers.se`

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

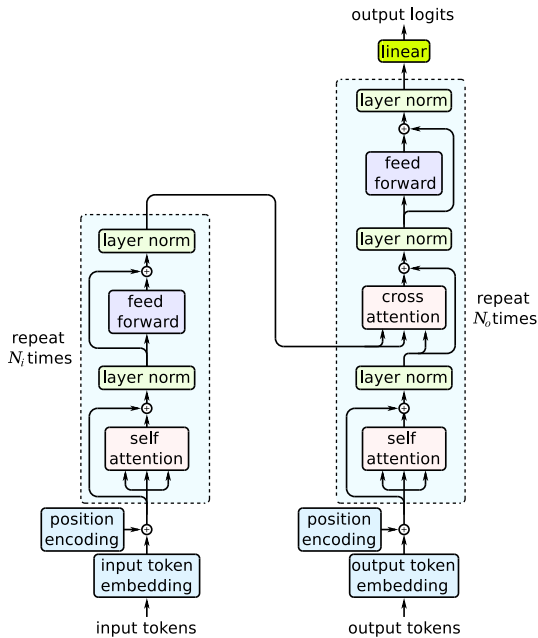
Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

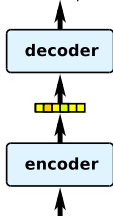
The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention



types of Transformers: high level

encoder/decoder:
(translator)

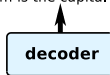
Stockholm is the capital of Sweden.



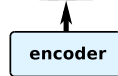
Stockholm är Sveriges huvudstad.

decoder only:
(GPT-style)

Stockholm is the capital of Sweden.

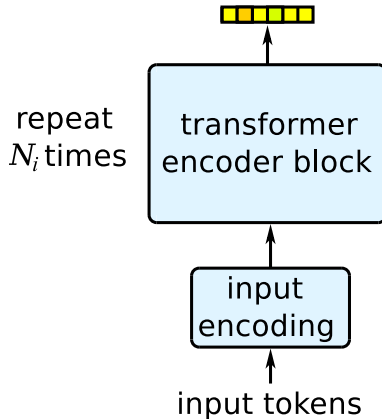


encoder only:
(BERT-style)



Stockholm är Sveriges huvudstad.

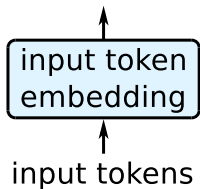
high-level overview of the Transformer encoder



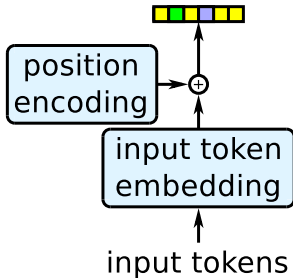
bottom-level token representations

input tokens

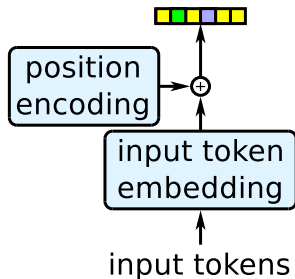
bottom-level token representations



bottom-level token representations

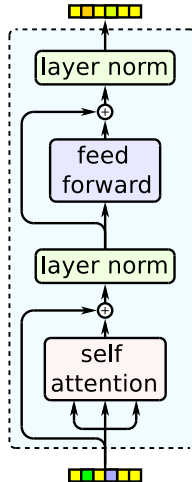


bottom-level token representations



- Vaswani et al. (2017) used a sine-based function to represent positions
- recent Transformers typically use **learned position representations**
- NB: this is the **only** place where position is taken into account

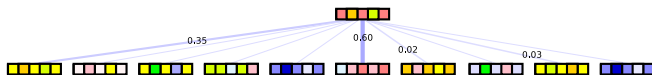
transformer **encoder** block: overview



attention models: recap

- first, compute an “energy” e_i for each position h_i
- for the **attention weights**, we apply the softmax:

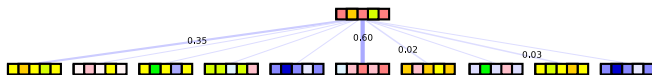
$$\alpha_i = \frac{\exp e_i}{\sum_{j=1}^n \exp e_j}$$



attention models: recap

- first, compute an “energy” e_i for each position h_i
- for the **attention weights**, we apply the softmax:

$$\alpha_i = \frac{\exp e_i}{\sum_{j=1}^n \exp e_j}$$



- finally, the “summary” is computed as a **weighted mean**:

$$\text{attention} = \sum_{i=1}^n \alpha_i h_i$$

attention as a soft lookup memory

attention can be thought of as a mapping from a **query** q and a set of **key-value pairs** k_j, v_j to an output

an attention weight α_j can be seen as an affinity between q and k_j

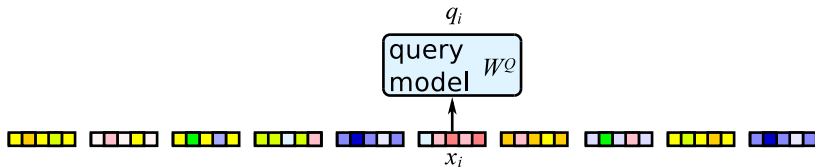
the output is the weighted mean of the **values**:

$$\text{attention} = \sum_j \alpha_j v_j = \alpha V$$

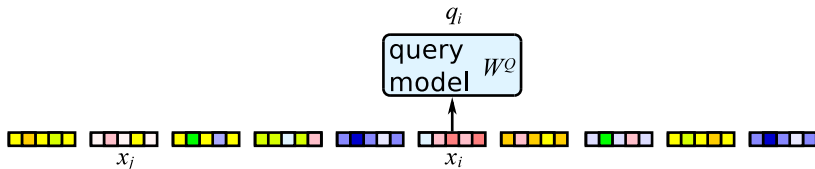
Transformer attention: computing attention weights



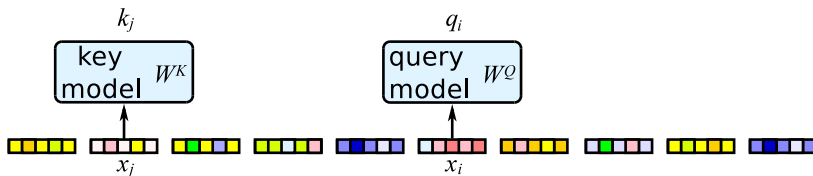
Transformer attention: computing attention weights



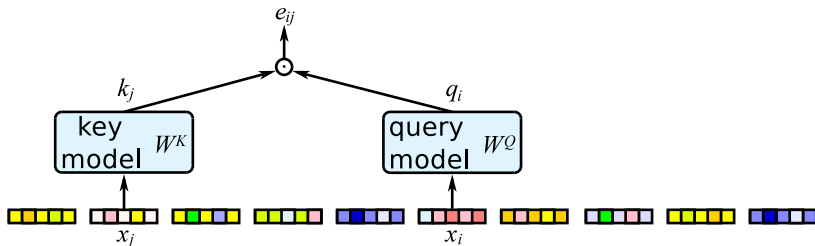
Transformer attention: computing attention weights



Transformer attention: computing attention weights



Transformer attention: computing attention weights



scaled dot product attention:

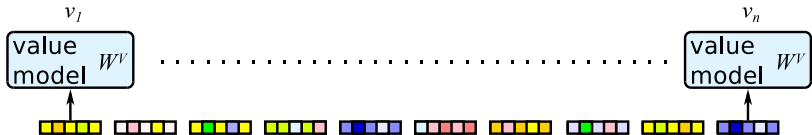
$$e_{ij} = \frac{q_i \cdot k_j}{\sqrt{d}}$$

Transformer attention: computing the weighted means



Transformer attention: computing the weighted means

compute **value representations** $V = W^V X$

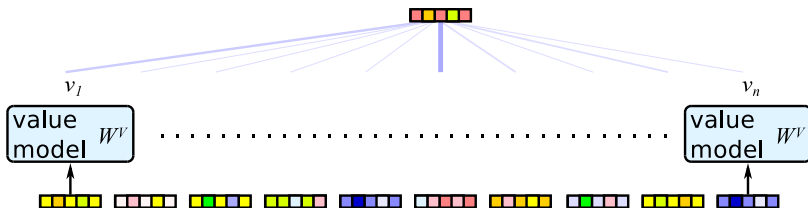


Transformer attention: computing the weighted means

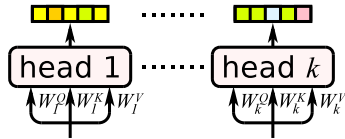
compute **value representations** $V = W^V X$

the attention output at position i is the weighted mean:

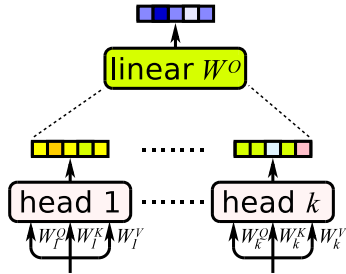
$$a_i = \alpha_i V$$



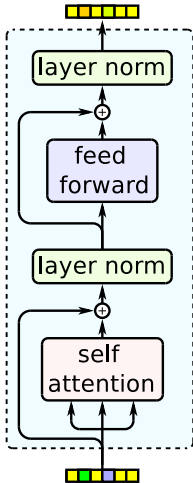
multi-head attention



multi-head attention



the rest of the encoder block

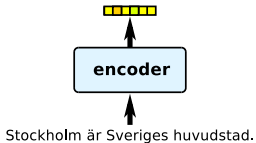


a **feedforward layer** is applied after MHA: it consists of a linear layer plus ReLU activation

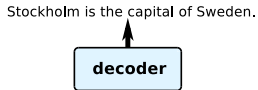
residual connections (He et al., 2016) and **layer normalization** (Ba et al., 2016) are used to stabilize training of deep models

recap

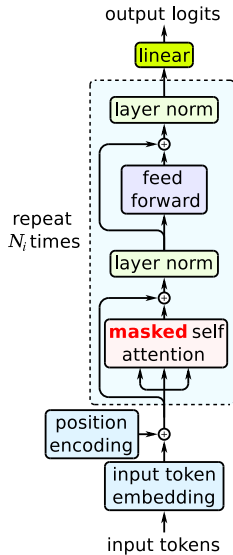
encoder only:
(BERT-style)



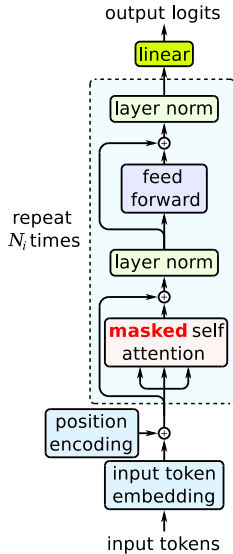
decoder only:
(GPT-style)

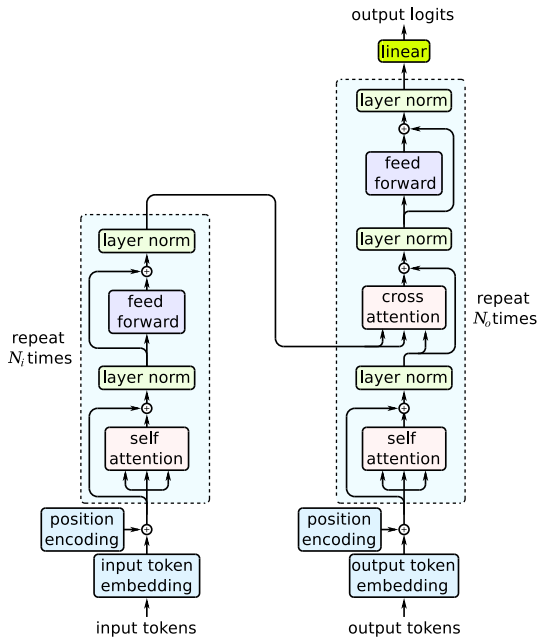


GPT-style decoder-only Transformer

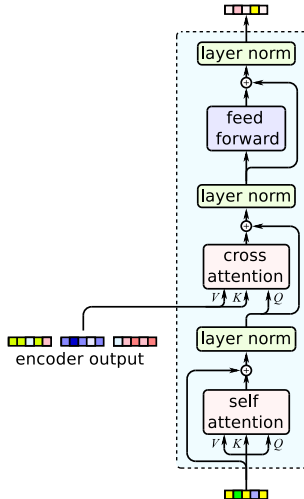


GPT-style decoder-only Transformer





a decoder block in an encoder/decoder Transformer



variations of Transformers

- Transformers are a **family** of models
- encoder/decoder or just encoder or decoder?
- how to deal with positions?
 - **hard-coded** position encoding? (Vaswani et al., 2017)
 - **learned** position embedding? (Devlin et al., 2019)
 - **relative** position embedding? (Shaw et al., 2018)
 - position in lowest layer or in all layers?
- layer normalization, other small details...

reading

The Illustrated Transformer

references

- J. Ba, J. Kiros, and G. Hinton. 2016. [Layer normalization](#). arXiv:1607.06450.
- J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. 2019. [BERT: Pre-training of deep bidirectional transformers for language understanding](#). In *NAACL*.
- K. He, X. Zhang, S. Ren, and J. Sun. 2016. [Deep residual learning for image recognition](#). In *CVPR*.
- P. Shaw, J. Uszkoreit, and A. Vaswani. 2018. [Self-attention with relative position representations](#). In *NAACL*.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. 2017. [Attention is all you need](#). In *NIPS* 30.